

CSV-Ressources for PCA

github.com/xylokopa

7. Juli 2026

1 Data-Ressources

- 1. Die Fahrzeug-Klassiker sind „Auto-mpg“ und „mtcars“. Mit Daten-Profil (Gewicht, PS, Hubraum, Beschleunigung) sind sie ein Standard in Data Science. Variablen wie Hubraum, Leistung und Gewicht korrelieren stark miteinander (perfekt, um die Varianz auf wenige Hauptkomponenten zu reduzieren).
- Variablen : mpg (Verbrauch), cylinders, displacement (Hubraum), horsepower (PS), weight (Gewicht), acceleration (Beschleunigung), model-year (Baujahr), origin (Herkunft).
- Web Link github.com/JWarmenhoven/.../Auto.csv
- Raw Dataset 1 <https://github.com/selva86/datasets.../Auto.csv>
- Raw Dataset 2 <https://github.com/selva86/datasets.../Vehicle.csv>
- Raw Dataset 3 [raw.data 24 features](#)
- Web Link [1974er MotorTrend-Datensatz als Gist CSV \(mtcars.csv\)](#)
- Raw Dataset <https://gist.githubusercontent.com/seankross/-etc-/mtcars.csv>.
- html-Dokumentation [mtcars.csv Feature-List](#)
- 2. Weitere Beispiele aus Geodaten, Weltbevölkerung grosse Datasets
- Beispiel Datensatz Wettersation: <https://mesonet.agron.iastate.edu/data/csv/coop.csv>
- World-Population 14 indicators 1962-2007: [14indicators1962-2007.csv](#)
- Stock-Market Ticker-Data: [Stock-Tickers.csv](#)
- For Python-Code: <https://www.kaggle.com/code/dgupta18/country-clusters>.
- For csv-Download: [unsupervised-learning-on-country-data](#) oder [unsupervised-learning-on-country-data?resource=download](#)
- Geografische Daten (Breiten- und Längen) direkt über [google/canonical/countries_csv](#).
- Beispiele sehr grosser Datensätze: <https://github.com/datablist/sample-csv-files>
- Schnellstart mit Python und URLs oder csv per pandas ins PCA-Skript:

```

import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
# Steuerungsvariable: True fuer Internet, False fuer lokale Datei
online = True
# Datenquelle basierend auf der Variable 'online' auswaehlen
if online:
    car_url = "https://raw.githubusercontent.com/selva86/datasets/refs/heads/master/Auto.csv"
    df_cars = pd.read_csv(car_url)
else:
    df_cars = pd.read_csv("auto.csv")
# Numerische Spalten fuer PCA auswaehlen ? korrigieren und
# standardisieren
df_cars['horsepower'] = pd.to_numeric(df_cars['horsepower'].replace('?',
    pd.NA), errors='coerce')
features = ['cylinders', 'displacement', 'horsepower', 'weight', '
    acceleration']
X = StandardScaler().fit_transform(df_cars[features].dropna())
# PCA berechnen
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print(f"Erklaerte Varianz der ersten beiden Komponenten: {pca.explained_
    variance_ratio_.sum():.2%}")

```

2 more ftp , Links and Lectures around Python and Data

1. **Python:** Stellenbosch University <https://ftp.sun.ac.za/ftp/pub/mirrors/python/>
2. **py-Index:** Python-Index <https://docs.python.org/3/py-modindex.html>
3. **gallery:** Matplotlib <https://matplotlib.org/stable/gallery/index.html>
4. **github.io:** Python DataScience [Python-Handbook](#)
5. **pub latex:** ftp.dante.de [mirror](#)
6. **pub math:** Laboratoire LIP6 <https://ftp.lip6.fr>
7. **py pdf:** SMART PARKING SYSTEMS <https://cdn.aui.ma/sse-capstone-repository>
8. **py math:** F.F.T-Implementation [towards data science](#)
9. **py math:** Fast Fourier Transform in Climate- and Geo-Physics <https://kls2177.github.io>
10. **py math:** FFT with Scipy <https://docs.scipy.org>
11. **py PCA:** World Bank Data in Human Fertility <https://www.statsmodels.org>
12. **py PCA:** What is PCA-Analysis ? <https://python-course.eu>
13. **py PCA:** Scree-Plots explained in Orange <https://orange3.readthedocs.io>
14. **py PCA:** Github-Notebook-Examples <https://github.com/erdogant/pca>
15. **py PCA:** SciKit- about PCA <https://scikit-learn.org>
16. **py PCA:** Examples in R Markdown by Paul Jozefek for PCA <https://rpubs.com/pjozefek>
17. **py PCA:** PCA around the Covid-Gap <https://fam.tuwien.ac.at>
18. **py PCA:** Covid-2020 on Github <https://github.com/lukysummer>
19. **py PCA:** Pythonization of Covid 19 on RPubs <https://rpubs.com/hansdeck>

20. **py Data:** US Covid19 Dataset <https://raw.githubusercontent.com>
21. **py Data:** Global Covid19 Dataset <https://raw.githubusercontent.com>
22. **py Data:** Covid19 Aggregated Data <https://raw.githubusercontent.com>
23. **py Data:** Application Example <https://opensource.com/article>
24. **py Data:** yahooFinance Ticker-Examples <https://finance.yahoo.com/quote/SPCX/>
Speziell bei SpaceX sieht man den Sekundenticker und darunter Handelsvolumina.

```
# AAPL_Beispiel.py 50Zeilen Aktiensparen zwischen 2019 und 2021
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import datetime
import os
csv_Nam = "AAPL.csv" # geht von 2015 bis 2026,boersentaeglich!
if os.path.exists(csv_Nam):
    print(f"lokale csv {csv_Nam} wird geladen...")
df_csv = pd.read_csv(csv_Nam)
df_csv['Date'] = pd.to_datetime(df_csv['Date'])
df = df_csv.sort_values('Date').reset_index(drop=True)
df.set_index('Date', inplace=True)
df_aktiensparen = df.loc['2019-01-01':'2021-10-01'].copy()
df_monthly = df_aktiensparen.resample("MS").first()
df_monthly = df_monthly.dropna(subset=["Price"])
investment_per_month = 20
raten_plan = []
raten_nr = 0
for date, row in df_monthly.iterrows():
    price = row["Price"]
    units = investment_per_month / price
    datum = date.strftime("%d.%m.%Y")
    rate = round(units,6)
    raten_nr = raten_nr + 1
    print(raten_nr,datum,rate)
    raten_plan.append((rate))
gesamte_anteile = sum(raten_plan)
letzter_preis = df_monthly["Price"].iloc[-1]
depot_wert = gesamte_anteile * letzter_preis
# x-werte entspr. monatlich
x_zahlen = np.arange(len(df_monthly))
y = raten_plan
x_labels = [date.strftime("%d-%m-%y") for date in df_monthly.index]
plt.figure(figsize=(15, 6))
plt.plot(x_zahlen,y,label='Inv.20Eu/mtl',color='red',linestyle='solid')
plt.xlim(0, len(df_monthly) - 1)
plt.xticks(ticks=x_zahlen[::2], labels=x_labels[::2], rotation=0)
titel_text = (f"AAPL(2019-2021)\n")+(f"Depot:{depot_wert:.2f}Eu")
            +(f"Anteile: {gesamte_anteile:.4f} Stueck")
plt.title(titel_text, fontsize=12, fontweight='bold')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.tight_layout()
plt.show()
```